



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Data replication

Michele Colajanni

Dipartimento di Informatica – Scienza e Ingegneria
michele.colajanni@unibo.it

Replication of server machines

One blade



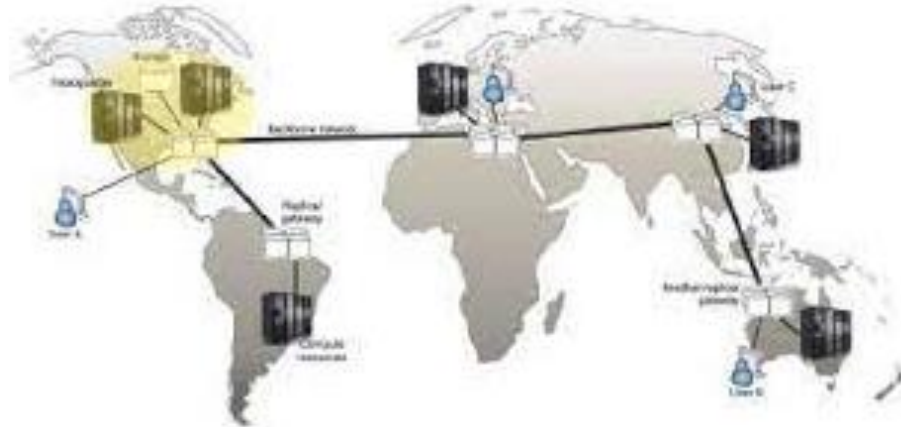
Multiple blades = Rack



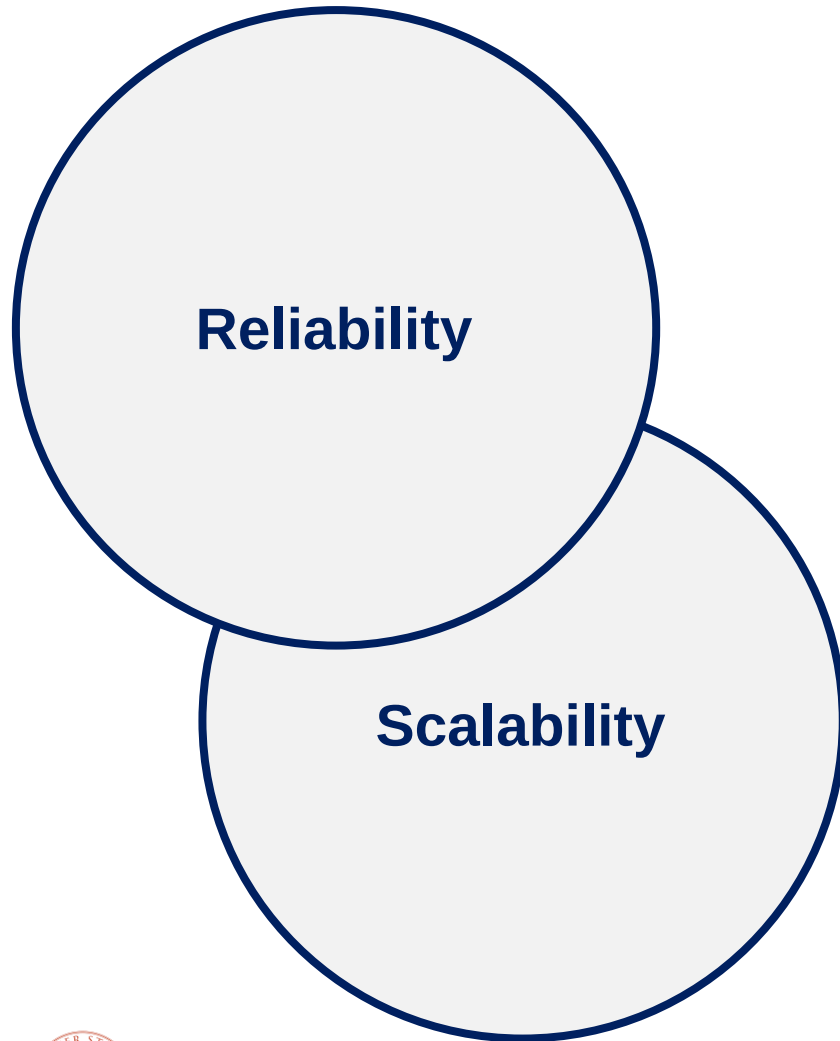
Multiple racks = IDC
(local replication)



Multiple IDCs
(geographic replication)



Replication: *not only the architecture*



Replication of server machines

Replication of interconnections

Replication of data

Replication of application processes

Replication of people



Don't forget

**Data are not important.
They are the only thing that
matters in the digital society!**



Data replication

Several alternatives exist among replicated copies:

1. Just for ***reliability*** or even for ***scalability***
2. ***Strong*** or ***weak*** data consistency
3. ***Full*** or ***Partial replication***

Why?

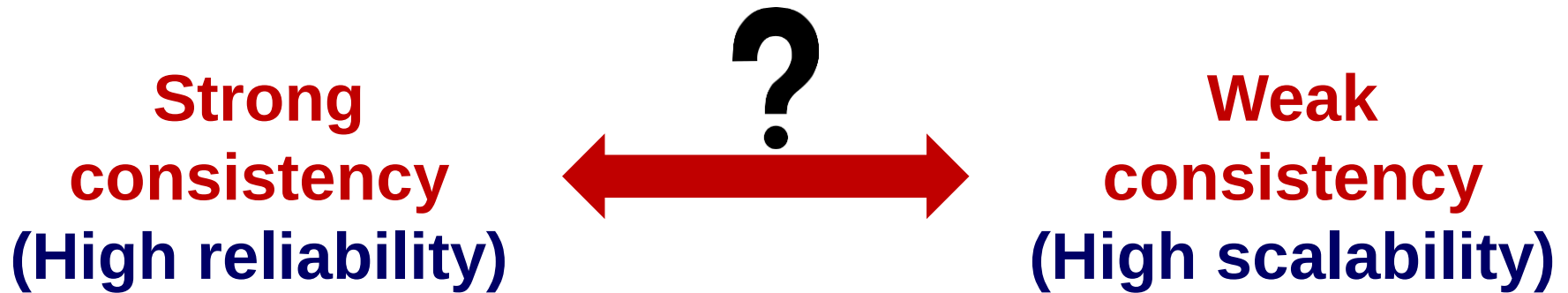


Because data replication has an intrinsic cost that impacts on performance and scalability

***Synchronization overhead
for
data consistency***



Engineering tradeoff: “You must pick one of the two” or ...



Look for if and what compromise is possible



The nature of the application drives the compromise

- Strong consistency penalizes performance and scalability, but it is necessary for some applications
 - For other applications there is a large spectrum of weaker synchronization alternatives
- ➔ It is our duty to explore the best compromise

Strong

W e a k

No consistency



Features that must be considered

- 1) **Data synchronization**
- 2) **Concurrency control system**

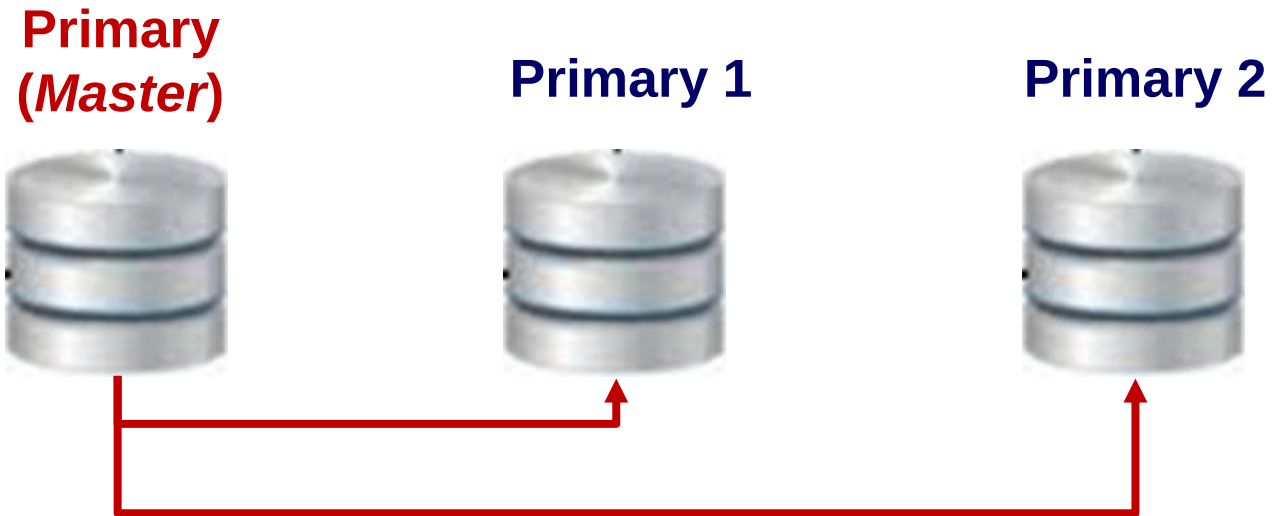


1) Data synchronization

- Data replication is an effective measure for scalability and reliability only if:
 - the service guarantees a single image to users (that are and must be kept unaware of the complexity behind data replication management)
 - the system guarantees strict consistency among all data replicas
- On the other hand, **inconsistency** between replicas hurts the quality property of the service



Strong data synchronization: all copies are primary



2) Concurrency control system

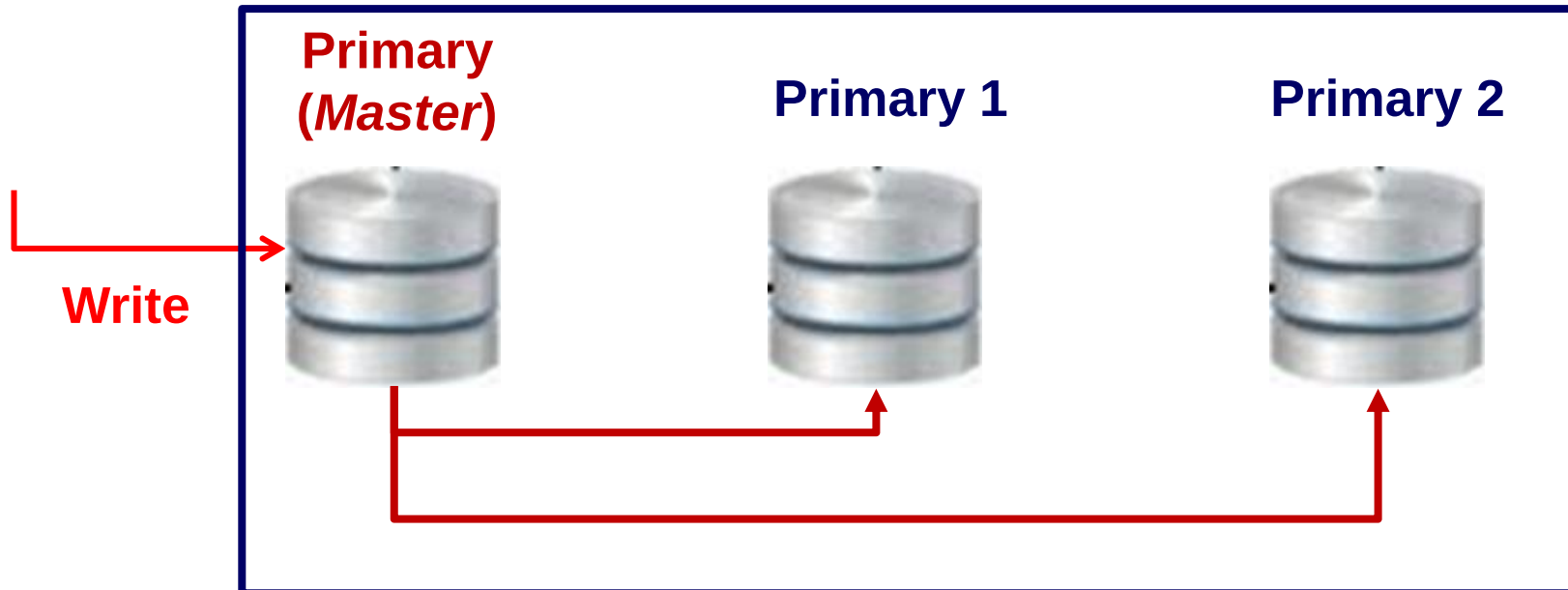
- **Data replication must be transparent to users**
- The possible parallel execution of transactions on a replicated system must be equivalent to a serial execution of those transactions on a one-copy system

➔ **One copy serializability**

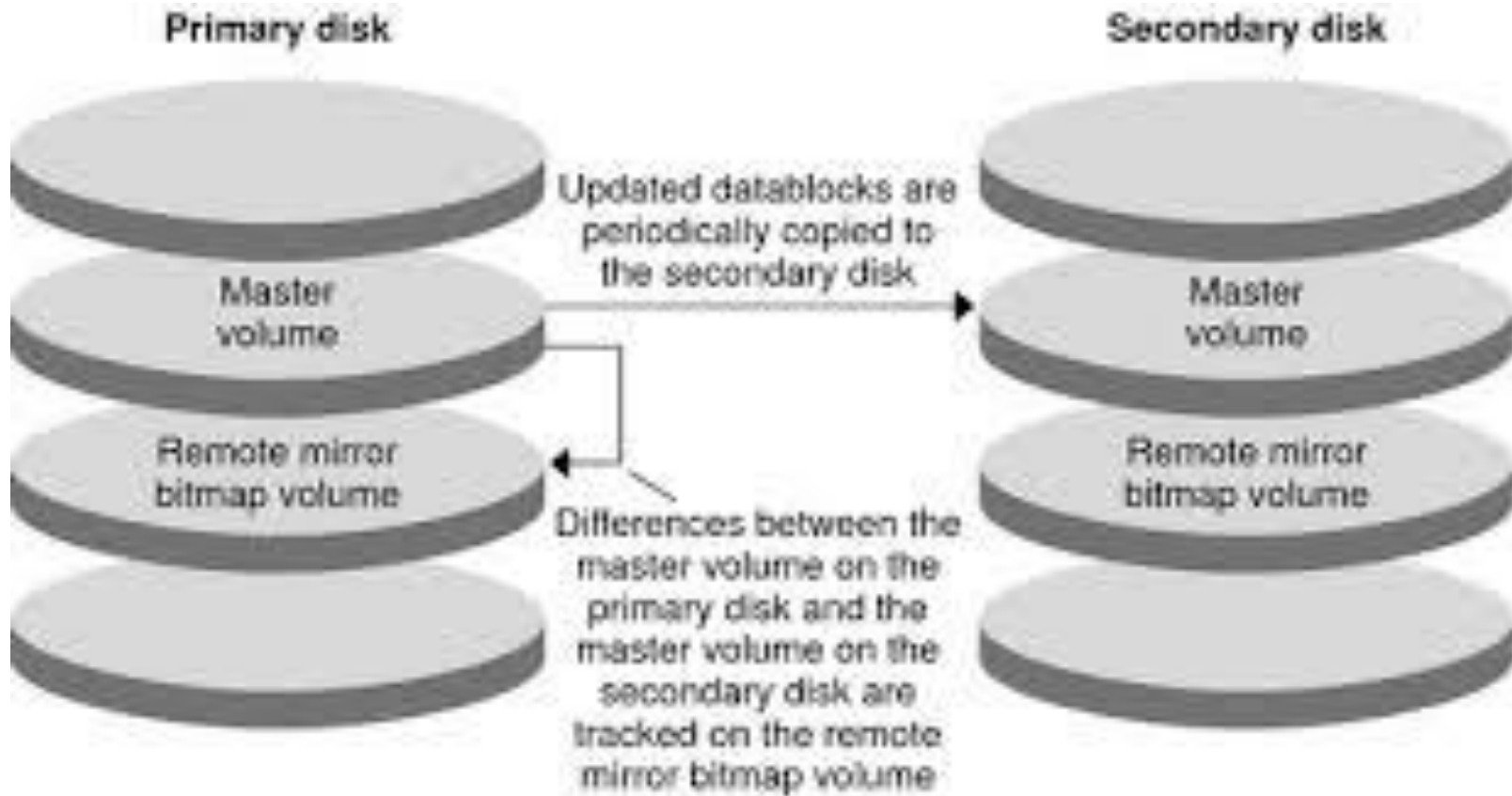


One copy serializability

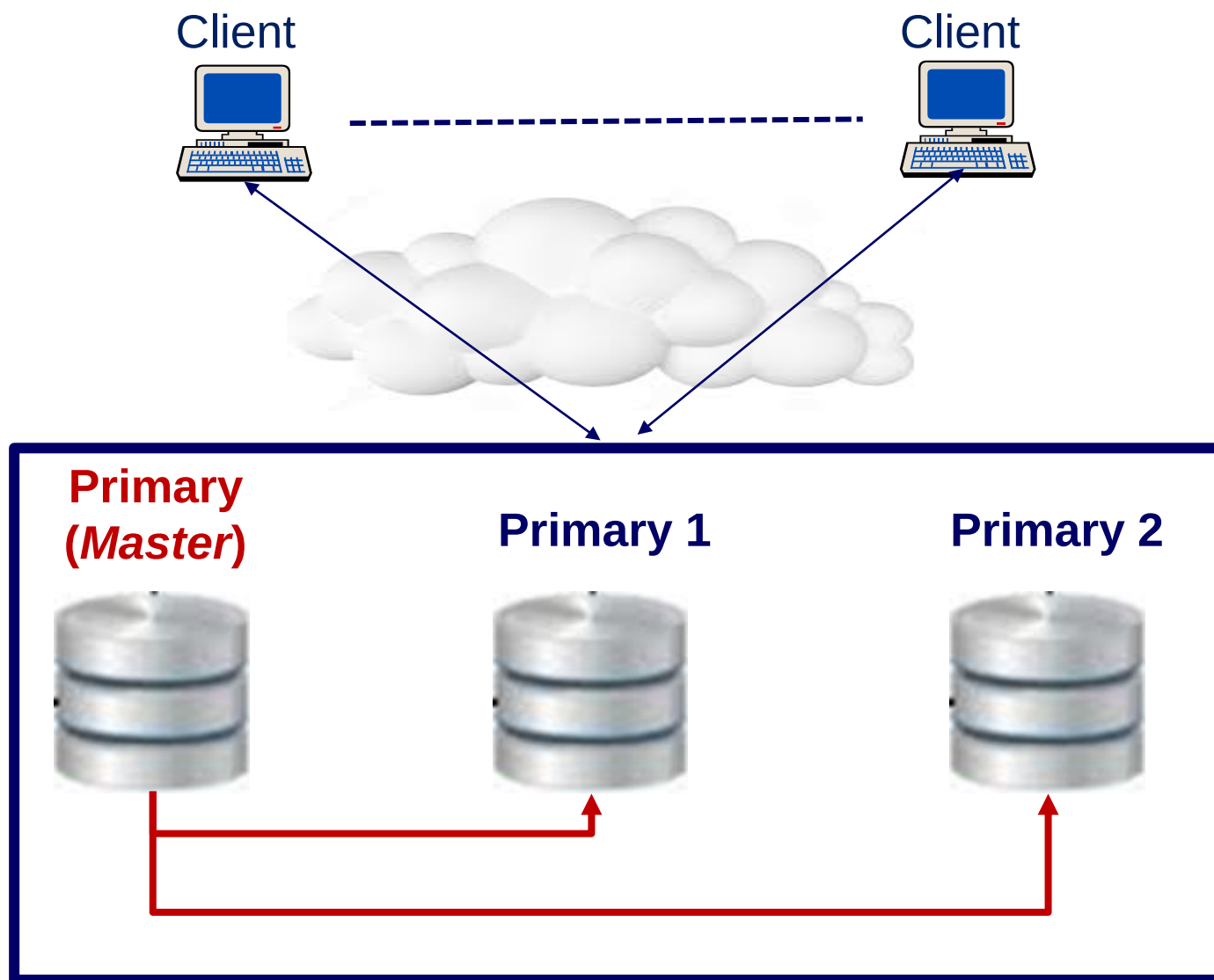
Consistency level: The *concurrency control system* allows the successive operation only after synchronization



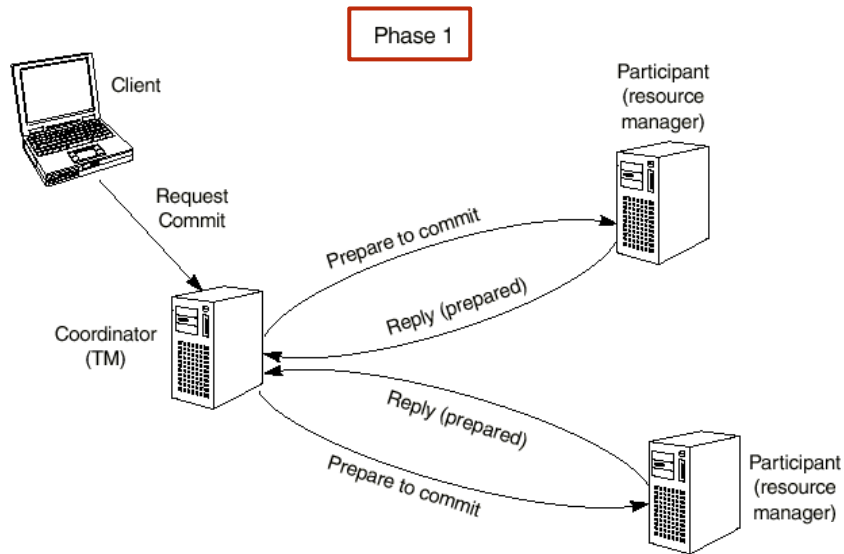
One copy serializability: hardware/OS solution (e.g., adopted in RAID disks)



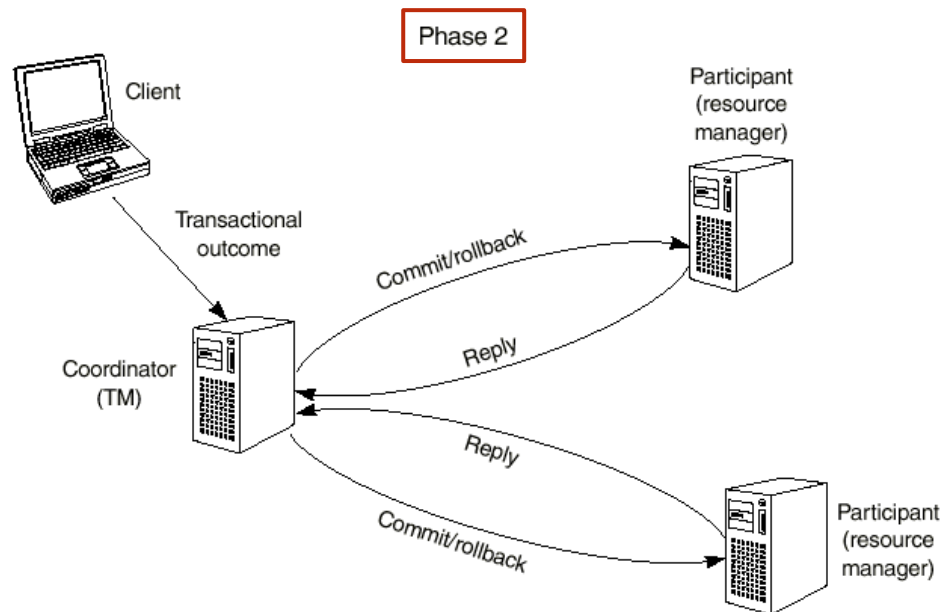
One copy serializability: application-based solutions



Protocol for strong consistency: *Two-phase commit* (typically for locally distributed copies)



**Built for the synchronization
of distributed databases**
(Expensive in performance terms)



Can data consistency be achieved even among geographically distributed copies?



The answer is **YES**

The second question is: *Which type of consistency is acceptable for which application?*



Alternatives

1. Strong consistency

- **Two-phase commit protocol** (*if you have a fast WAN connection*)

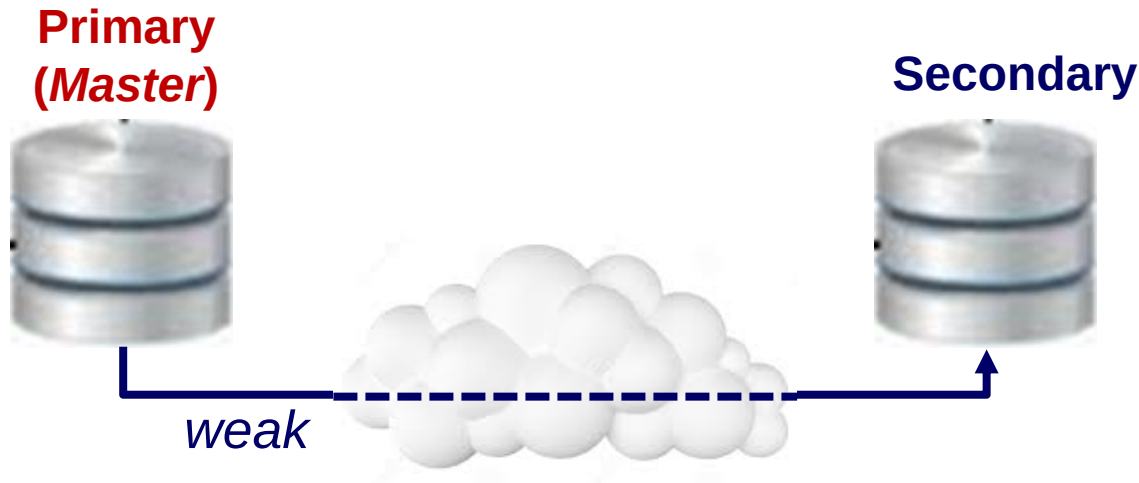
2. Weak consistency including an entire spectrum of semantics

- **Primary and secondary copies**
- **Gossip protocols**
- **Each service implements its own solution for geographically distributed data.** There is a large design space for such mechanisms: different solutions may be optimal for different types of data and services
 - *Facebook* replicates data with all writes going through one primary datacenter
 - *Yahoo! mail service* partitions data across datacenters based on user account



Backup for disaster recovery

- The secondary (*backup*) copy is synchronized periodically
 - the period can go from few minutes to one day and, in some case, even beyond
- The secondary copy is used only when necessary



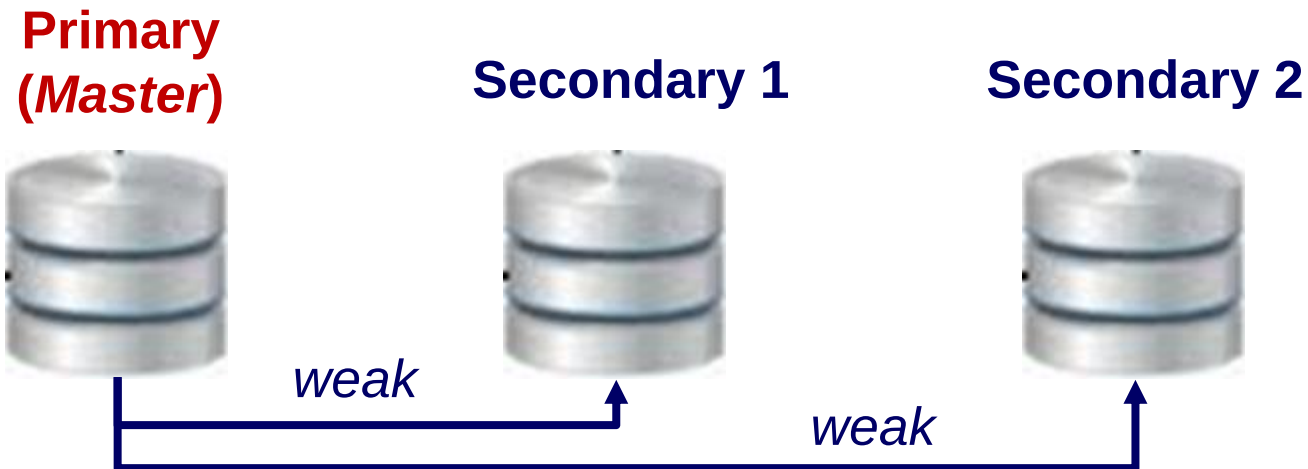
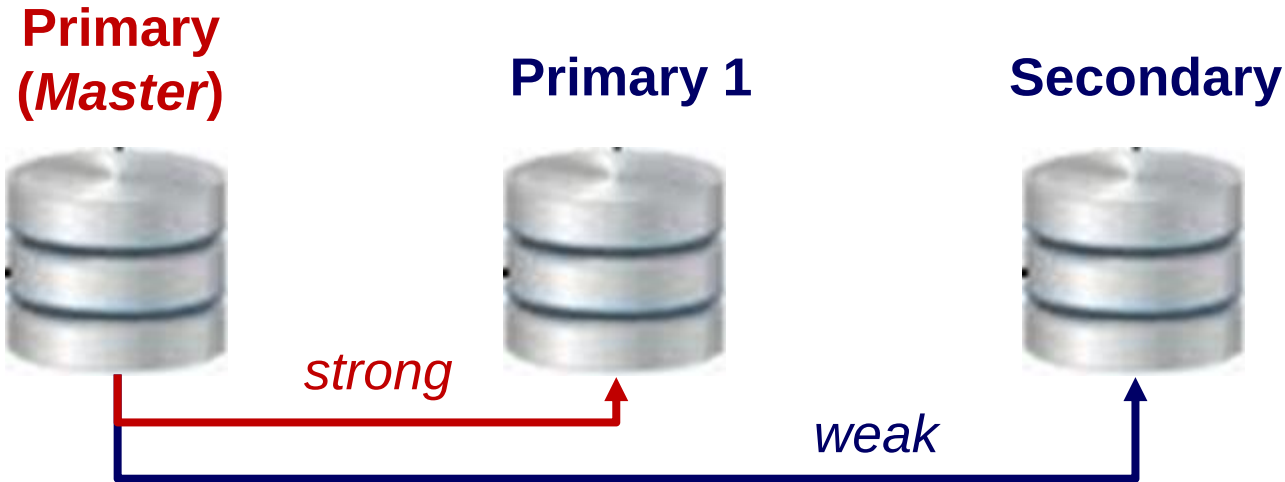
- Italian law for the public administration requires at least 30Km of distance and infrastructure (power, network, etc.) independence [Availability best practice indicates much more than 30Km: different states, different continents, the cloud]

Primary and secondary servers

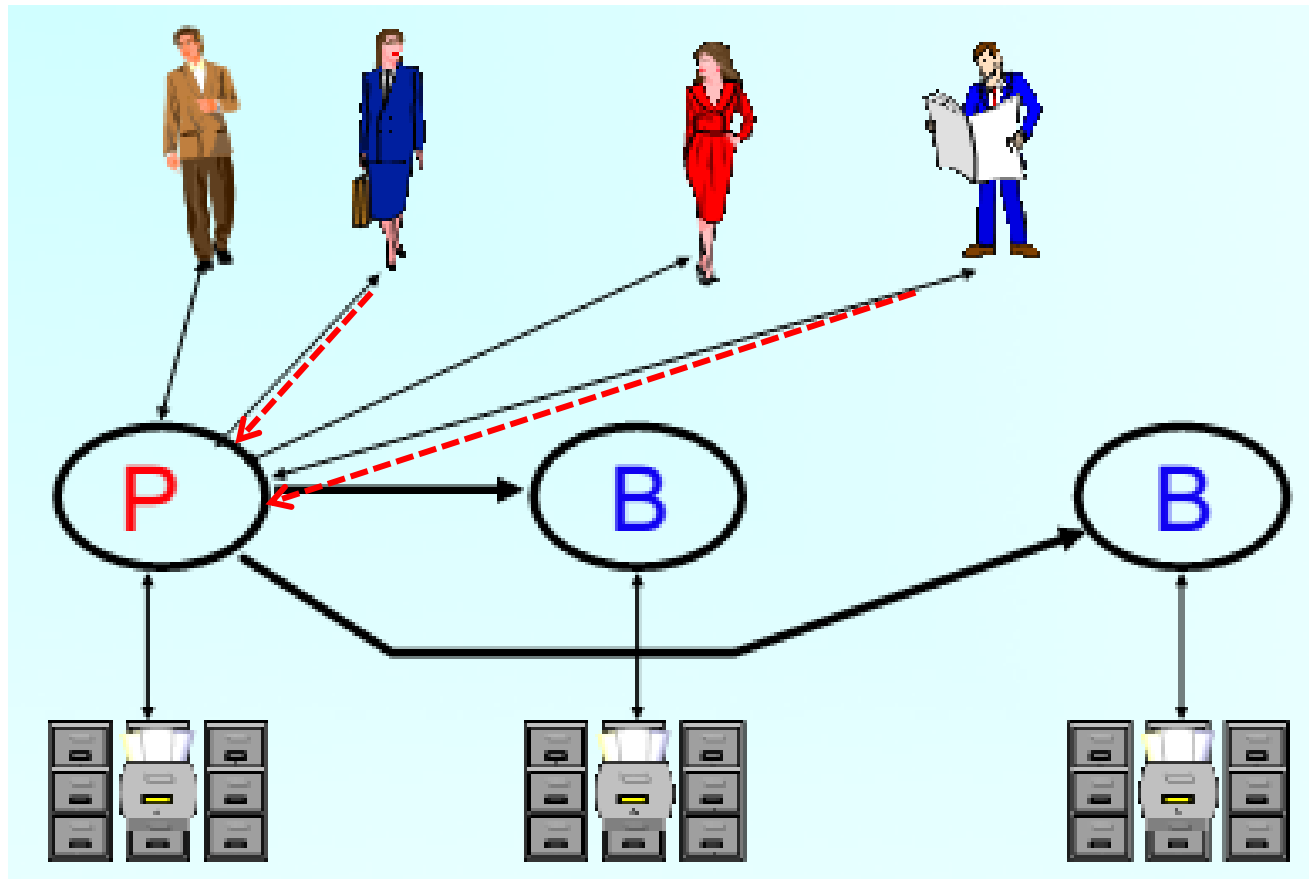
- Secondary servers are initially designed and maintained for reliability purposes
- However, secondary servers can also be used to improve scalability for any **read operations**, while **update operations** must be sent to the primary server and then it is synchronized with its secondary server(s)



Strong and weak synchronization: *copies may be primary or secondary*



Example 1: All operations go to the Primary

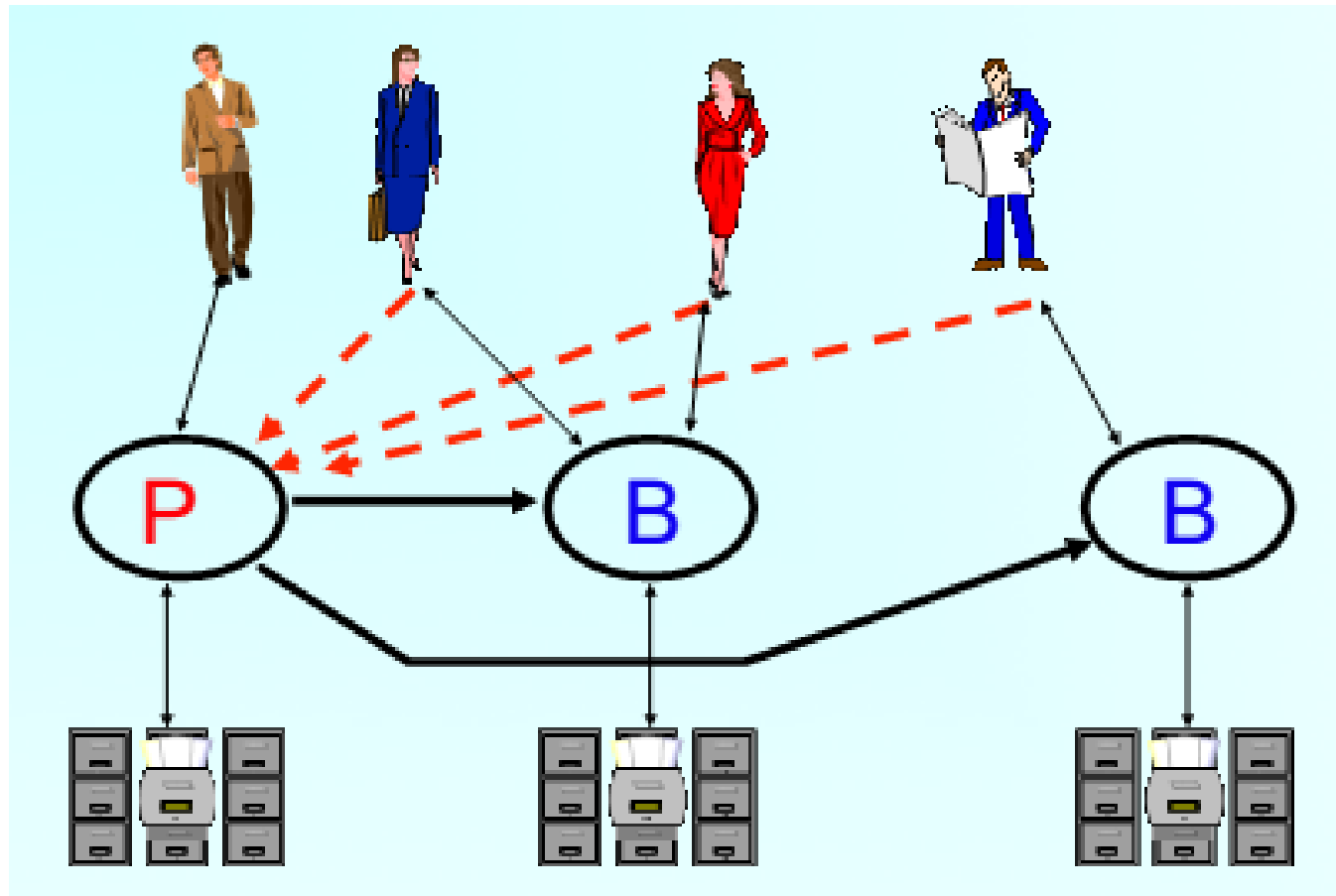


-----> **Write operations**

————> **Read operations**



All *Write* operations go to the Primary- *Read* operations go to any server (if they are “well” synchronized)



-----> **Write operations**
—————> **Read operations**



Example of alternatives

- Data for a *buddy list* (e.g., list of friends) and *buddy status* (e.g., online or not) fit naturally to a model where information is replicated between distributed datacenters with **weak consistency assurances**
- On the other hand, an **email service** maps naturally to a model where each collection of data (that is, a mailbox) is accessed by a single user, and the data can be partitioned across multiple datacenters by user identification, thus requiring **strong consistency assurances**



Weak consistency

State kept by the secondary servers can be weakly consistent (copies are allowed to diverge temporarily) and they will eventually come to an agreement

- From time to time, a server picks another server and these two servers exchange updates and converge to the same state
- Total ordering is obtained after getting one message from every server (*directly*) → **Lamport time-stamps are used to order messages**

The first paper on this topic was published in 1992, but many Internet-based applications now leverage it for scalability purposes (e.g., Google Hadoop, Facebook)



Modern approach to scalability: NoSQL

- **Non relational databases** (*hundreds...*)
 - MongoDB, CouchDB, Elastic search, ...
 - Neo4j, OrientDB, ... (graph DB)
 - BigTable, Velocity, Reddit, Cassandra (key-value DB)
- **Attributes**
 - High scalability
 - Large-scale distribution
- **What do we pay in exchange?**

Transactional consistency

In contrast to ACID systems that enforce consistency at transaction commit,
NoSQL guarantees consistency only at some undefined future time



NoSQL: BASE vs ACID

- **RDBMS** → **ACID** properties
 - Atomicity
 - Consistency (hard requirement)
 - Isolation
 - Durability
- **NoSQL** → **BASE** properties
 - **B**asically **A**vailable
 - **S**oft state
 - **E**ventually consistent



BASE properties

- **Basically Available:** Use data replication to reduce the likelihood of data unavailability and partition the data among many storage servers. The result is a system that is always available, even if subsets of the data may become unavailable for short periods of time
- **Soft state:** NoSQL systems allow data to be inconsistent and relegate the design of services around such inconsistencies to application developers
- **Eventually consistent:** Although applications must deal with instantaneous consistency, NoSQL systems ensure that -at some future point in time- data will assume a consistent state



Inventors and early adopters?

- Oracle
- IBM
- Microsoft
- Sleepycat Software
- Amazon
- Google
- LinkedIn
- Twitter



Inventors and early adopters?

- Oracle
- IBM
- Microsoft
- **Sleepycat Software**
- Amazon
- **Google**
- LinkedIn
- Twitter

→ Berkeley DB”

→ “Big Table”



Motivations

NoSQL emerged as a key technology when cloud companies struggled to deal with unprecedented data and operation volumes, millions of concurrent download and upload operations under tight latency constraints

- Analyzing in real-time high-volume data (e.g., Web site click streams, uploads, comments, profiling, preferences, advertisement) and accepting *User Generated Content* provides significant business advantages
- However, it requires **novel distributed data structures and synchronization strategies** to leverage structured, unstructured and semi-structured data sources



Modern data management

Operational Database Management Systems (ODBMS) include:

- In-Memory databases
- Graphs databases
- NewSQL databases
- NoSQL data stores
- Big Data management
- Analytical data platforms
- Cloud platforms
- Object-relational bindings
- ...



Geographical distributed services

- The state-of-the-art is that every service implements its own solution for geo-distribution of data
- There is a very large design space for such mechanisms, and different solutions may be optimal for different types of data

Remember:

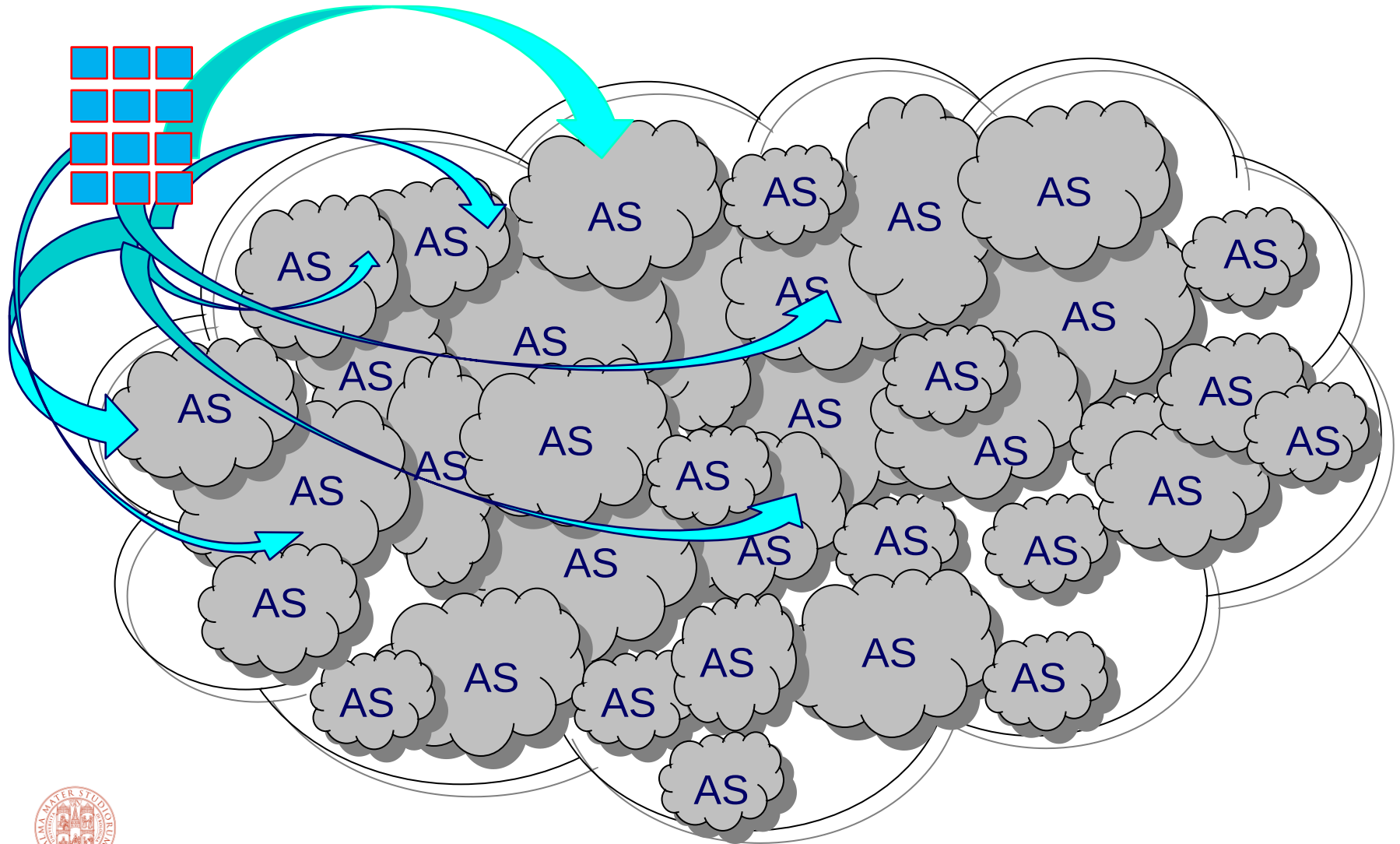
Strong

W e a k

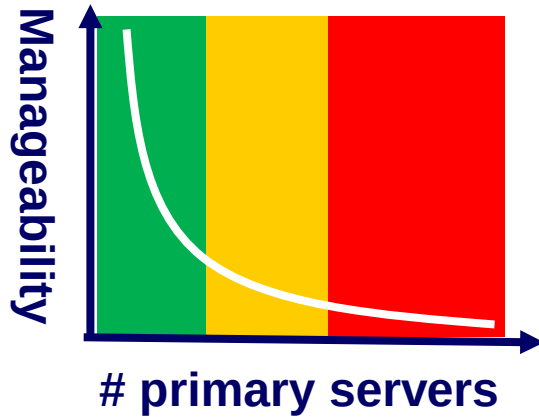
No consistency



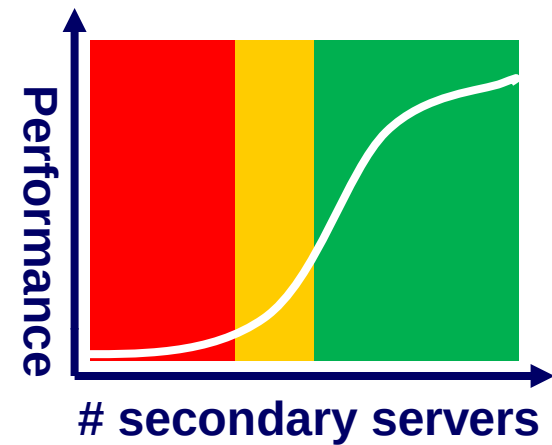
It is impossible to distribute primary servers in each Autonomous System



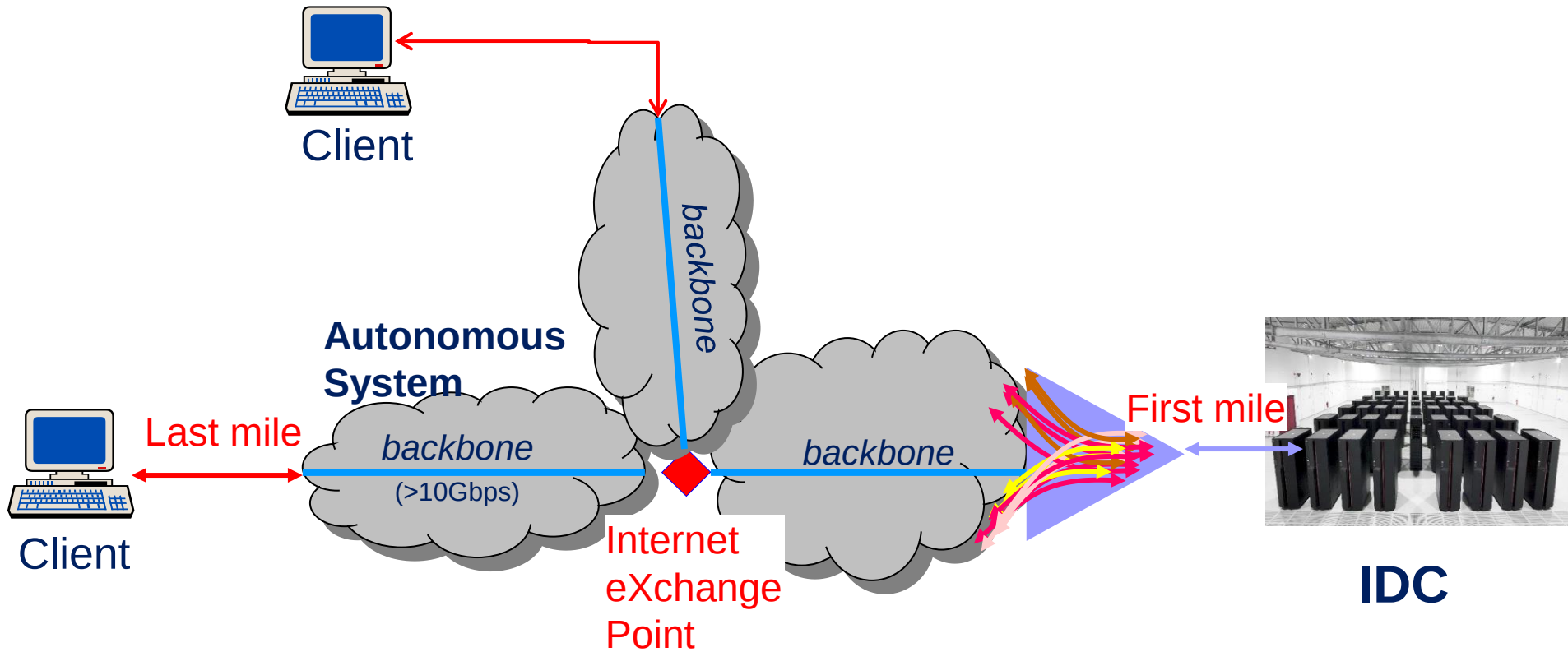
Motivation for *partial* replication



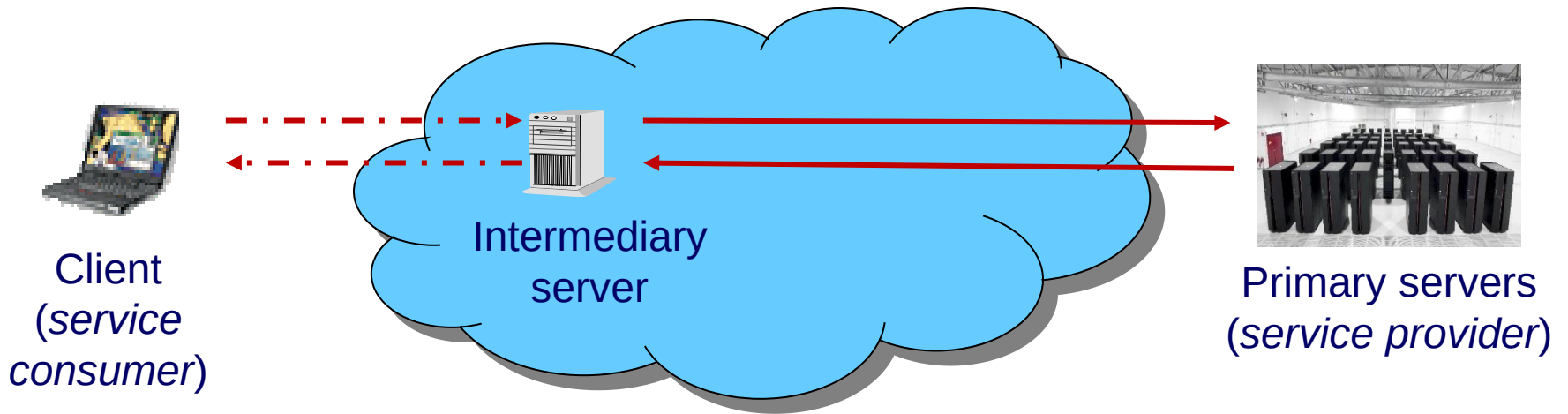
Alternative solution



Overall view



The basic idea



Many names – One concept

- Secondary servers
- Data servers
- Network cache server
- Cache server
- Proxy server
- Reverse proxy
- Edge server
- Edge POP
- ...

To replicate part of the origin content

Possibly in locations that are *closest* as possible to:

- the clients (if data transit from the servers through Internet)
- the servers (if data transit through Internet to the cloud)



Caching principles

- **Spatial locality:** if a user/application accesses a data, it is more likely that following accesses will be to “close” data instead of “far” data
- **Temporal locality:** if a user/application accesses a data, it is more likely that he/it will access again the same data in a “short” interval period instead of a “long” time



Efficacy of caching: measures

Caching is effective when the ways to access data by applications and users satisfy temporal and spatial locality

- **Cache hit**: the client finds the requested resource in the cache server
- **Cache miss**: the client does not find the requested resource in the cache server
- **Cache hit rate** (probability or percentage): number of cache hits in a time interval divided by the total number of requests in the same period of time



Caching mechanisms

- **Pull**: a content is uploaded from the primary server when a content consumer requests it, and there is no valid copy in the secondary server. Used by:
 - Proxy server
 - Systems of cooperative proxy servers (ISP)
- **Push**: contents that are requested with higher probability (or frequently accessed and changed) are pre-uploaded on secondary servers from the primary server of the content provider. Used by:
 - Reverse proxy
 - Content Delivery Network (CDN)



Data replication mechanisms

- **CONTENT CONSUMER**

- Proxy server

- **CONTENT PROVIDER**

- Web cluster + Reverse proxies
- Web cluster + proxies

- **THIRD PARTIES**

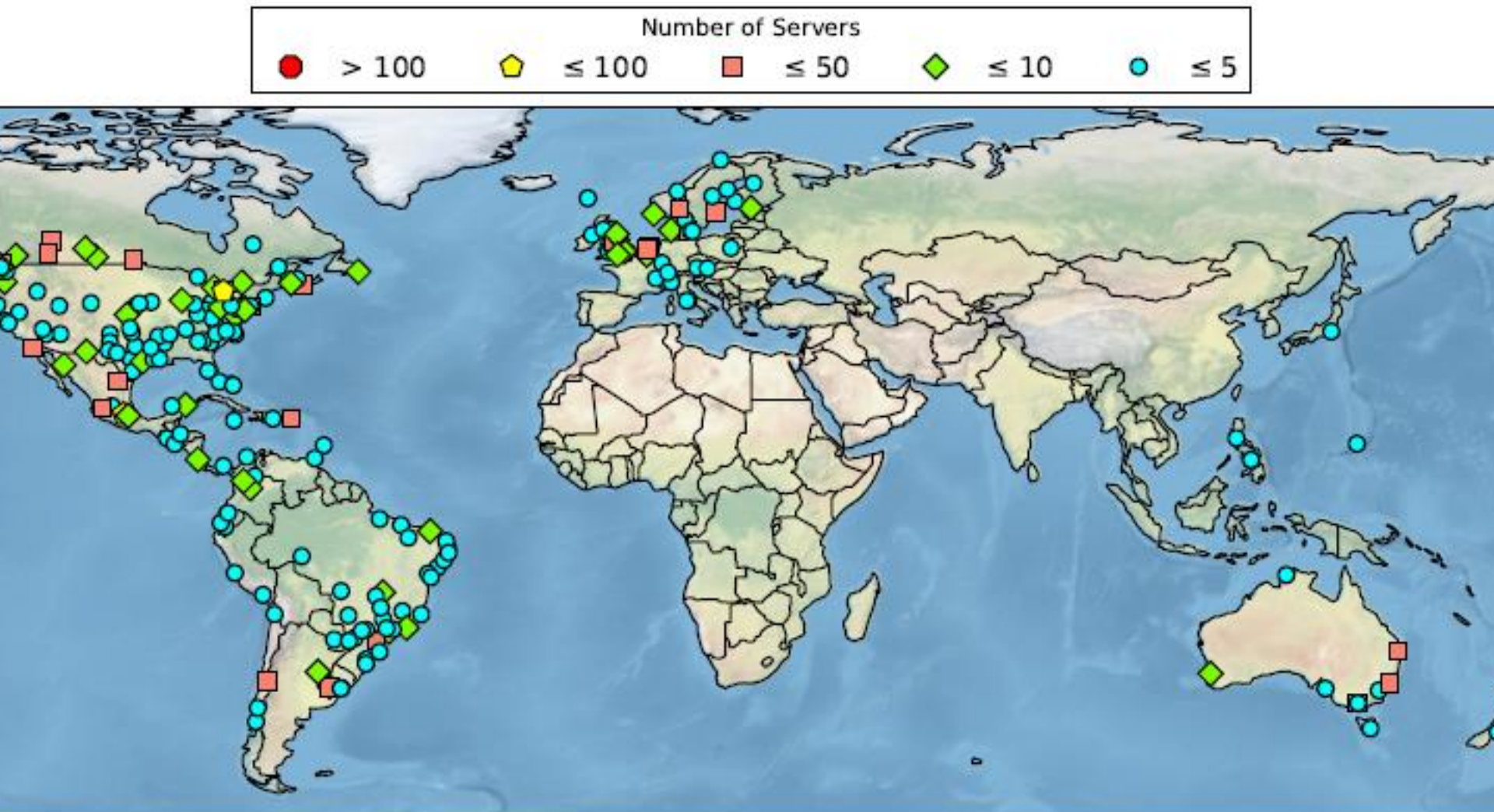
- ISP (*content consumer side*) → cooperative proxy servers
- Companies (*content provider side*) → Content Delivery Network (CDN)

ONE

MANY SERVERS



Geographical replication (Netflix servers)



Geographical replication (Google)

